

Theoretical Computer Science Exam, July 14th, 2017

The exam consists of **4 exercises**. Available time: 2 hr 00 min.; you may write your answers in Italian, if you wish.

All exercises must be addressed for the written exam to be considered sufficient.

F.NAME L.NAME MATRICOLA

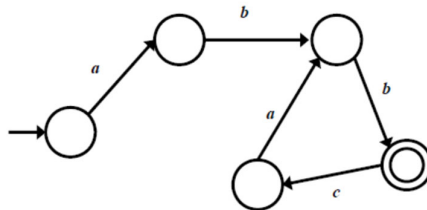
Exercise 1 (9 pts)

Given a language L over alphabet Σ , consider the following operation pref

$$\text{pref}(L) = \{ w \mid w \in \Sigma^* \text{ and } \exists x \in \Sigma^* \text{ such that } (wx \in L) \}$$

(notice that the result of $\text{pref}(L)$, for any language L , is also a language)

Consider the regular language L_1 accepted by the following finite state automaton.



1. Describe by means of an English sentence the language $\text{pref}(L)$ for a generic language L .
2. Write the first four strings of $\text{pref}(L_1)$ in lexicographical order.
3. Is $\text{pref}(L_1)$ a regular language? Please justify adequately your answer.
4. Is the family of regular languages closed under operation pref ? Please justify adequately your answer.

Exercise 2 (8 pts)

Write a monadic second-order logic formula over word structures that characterizes language L , of alphabet $\Sigma = \{a, b, c\}$ defined as follows $L = ab(bc)^+$.

Exercise 3 (8 pts)

Let us recall that a set S_1 is *reducible* (or, in other words, *it can be reduced*) to another set S_2 iff there exists a total computable function $t(x)$ such that, for every x , $x \in S_1$ if and only if $t(x) \in S_2$.

Now, in addition, we say that a set S_1 is *doubly reducible* to another set S_2 iff there exist two total computable functions $t_1(x)$ and $t_2(x)$ such that, for every x , $x \in S_1$ if and only if $t_1(x) \in S_2$ and $t_2(x) \notin S_2$.

Prove the following property, or disprove it if you think that it does not hold: if S_1 is reducible to S_2 and $S_2 \subset \mathbb{N}$ (i.e., S_2 is a proper subset of the universe set) then S_1 is doubly reducible to S_2 .

Exercise 4 (8 pts)

Design, preferably in full detail, a k -tape Turing machine that accepts the language over alphabet $\Sigma = \{a, b, c\}$

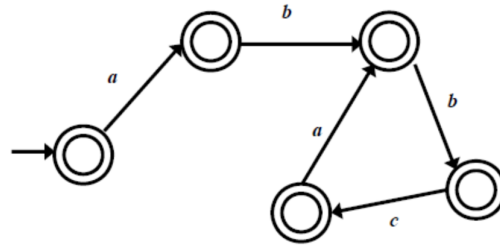
$$L = \{ wcw \mid w \in \{a, b\}^+ \} \cap \{ wcw^R \mid w \in \{a, b\}^+ \}$$

with minimal time complexity, and determine as precisely as possible its time and space complexity.

Solutions

Exercise 1

1. $\text{pref}(L)$ is the language of the *prefixes* (i.e., of the initial segments) of the strings of L .
2. The strings are: ε, a, ab, abb .
3. The language $\text{pref}(L_1)$ is obviously accepted by the following finite state automaton.



4. With a generalization of the previous answer, it can be argued that, given any finite state automaton A , one can obtain the automaton that accepts $\text{pref}(L(A))$ by making final all its states from which a final state of the original automaton can be reached; therefore the family of regular languages is closed with respect to *pref*.

Exercise 2

Write a monadic second-order logic formula over word structures that characterizes language L , of alphabet $\Sigma = \{a, b, c\}$ defined as follows $L = ab(bc)^+$.

$$\begin{aligned}
 & a(0) \wedge b(1) \wedge \exists X (X(2) \wedge \\
 & \quad \forall x ((x \geq 2 \rightarrow (b(x) \leftrightarrow X(x)) \wedge (c(x) \leftrightarrow \neg X(x))) \wedge \\
 & \quad \quad (X(x) \rightarrow \neg X(x+1)) \wedge (\neg X(x) \wedge \neg \text{last}(x) \rightarrow X(x+1)) \wedge \\
 & \quad \quad (\text{last}(x) \rightarrow \neg X(x)) \\
 & \quad) \\
 &)
 \end{aligned}$$

Exercise 3

The property holds. In fact, in the hypothesis that

- S_1 is *reducible* to another set S_2 (so that there exists a total computable function $t(x)$ such that, for every x , $x \in S_1$ if and only if $t(x) \in S_2$), and
- $S_2 \subset \mathbb{N}$ (so that there exists a $z \notin S_2$)

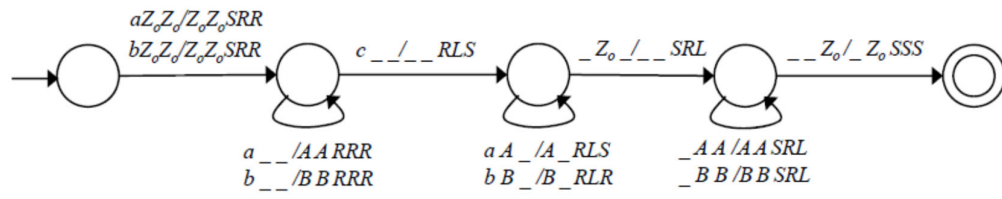
then it suffices to take as t_1 the function t above and as t_2 the total computable function $t_2(x) = z$ for all x .

Then the condition for S_1 being *doubly reducible* to S_2 is satisfied, because, for every x , $x \in S_1$ if and only if $t(x) \in S_2$ and $t_2(x) = z \notin S_2$.

Exercise 4

Notice that for a string $wcw \in L$ it holds $w = w^R$.

A 2-tape TM copies the part of the input before the c on both memory tapes, then it checks that the second part is the reverse of it by moving backwards on T1, then it checks that $w = w^R$ by moving forward on T1 and back on T2.



The time complexity for the Turing machine is $T_M(n) = \frac{3}{2}(n - 1) + 4$, while the space complexity is $S_M(n) = n - 1$.